

1. Introduction

The goal of this project was to implement and analyze graph algorithms, including Tarjan's Strongly Connected Components (SCC) algorithm, Condensation Graph construction, Topological Sorting, and Shortest Path in Directed Acyclic Graphs (DAG).

All implementations were tested on various input graphs of different sizes (small, medium, and large) represented in JSON format.

2. Implemented Classes

2.1 TarjanSCC

Implements Tarjan's algorithm to identify all Strongly Connected Components (SCCs) in a directed graph using depth-first search (DFS).

Each node is assigned a discovery time and a low-link value to detect cycles efficiently.

Key methods:

- `run(Graph graph)`: runs Tarjan's algorithm on a given graph.
- `getSCCs()`: returns a list of SCCs found.
- `getElapsedMs()`: returns the algorithm execution time.

2.2 CondensationBuilder

Builds a Condensation Graph, where each SCC from Tarjan's algorithm becomes a single vertex.

Edges between different SCCs are preserved from the original graph, forming a DAG.

Key methods:

- `build(Graph graph, List<List<Integer>> sccs)`: constructs the condensation DAG.

- `getCondensationGraph()`: returns the resulting DAG.

2.3 TopologicalSort

Implements Topological Sorting of the Condensation Graph using Depth-First Search (DFS).

It produces an ordering of vertices such that for every directed edge ($u \rightarrow v$), u appears before v in the order.

Key methods:

- `sort(Graph graph)`: returns a topologically sorted list of nodes.
- `getElapsedMs()`: returns the execution time.

2.4 DAGShortestPath

Computes the Shortest Path in a Directed Acyclic Graph (DAG) using relaxation in topological order.

This method works only for acyclic graphs and is more efficient than Dijkstra's algorithm for DAGs.

Key methods:

- `findShortestPaths(Graph dag, int source)`: computes minimum distances from the source node.
- `getElapsedMs()`: returns the computation time.

3. Test Cases

Unit tests were written for each module using **JUnit**:

Class	Purpose
TarjanSCCTest	Validates correctness of SCC detection
TopologicalSortTest	Checks correct topological ordering
DAGShortestPathTest	Verifies shortest path calculations in DAGs

All tests passed successfully on sample graphs.

4. Experimental Setup

Graphs were stored in JSON format (e.g., small1.json, medium2.json, large3.json) with fields:

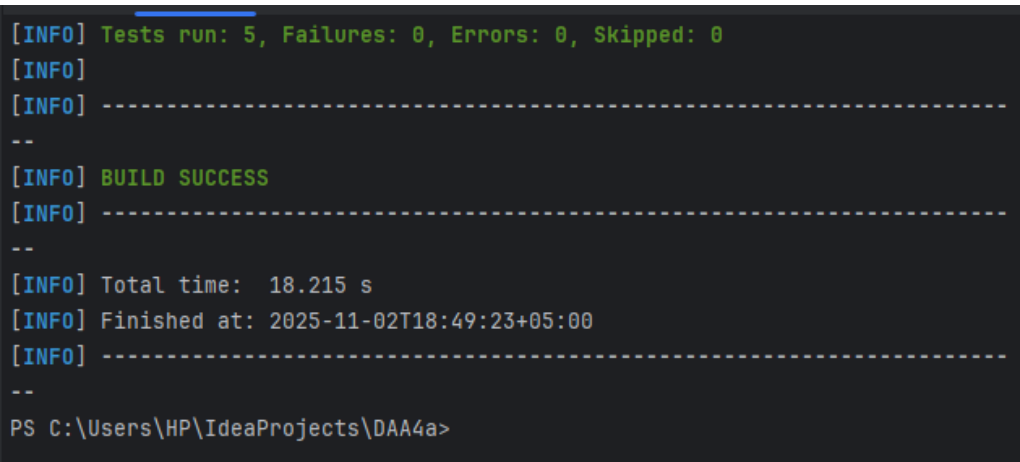
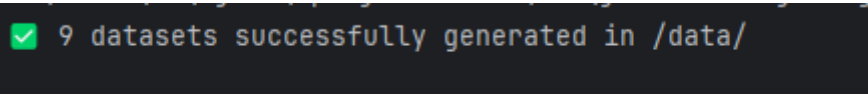
Size	Dataset	Nodes	Edges
Small	small1-3	6-9	7-10
Medium	medium1-3	15-20	23-35
Large	large1-3	30-50	60-130

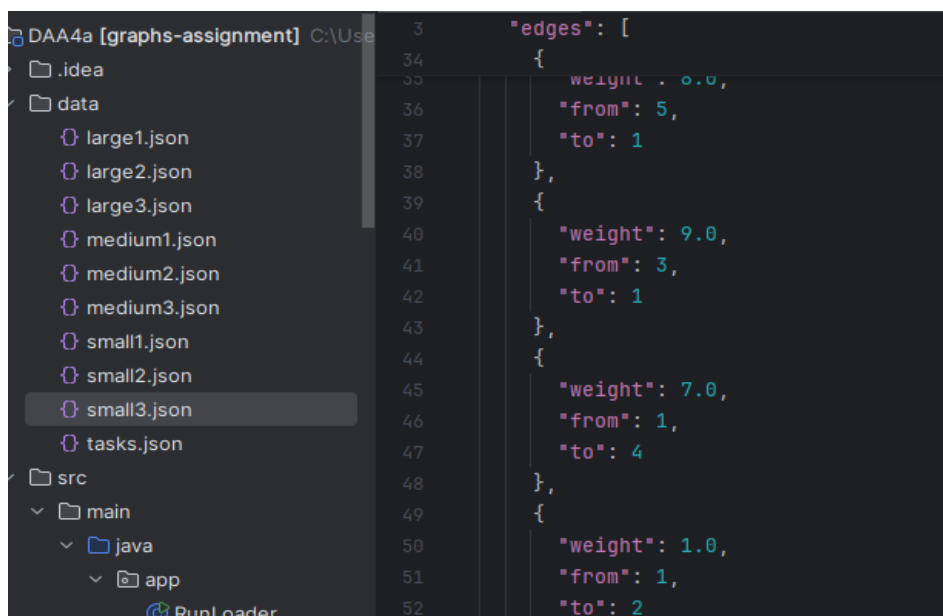
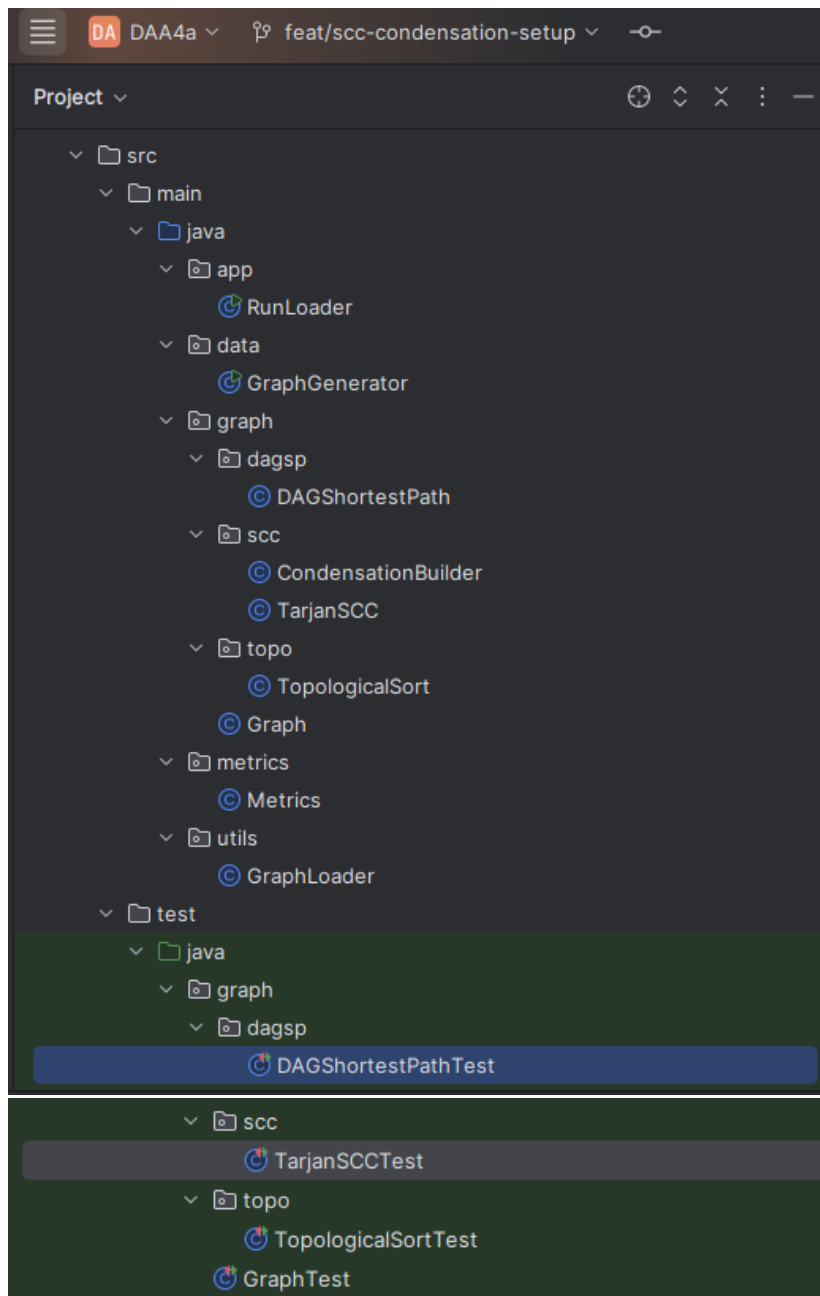
5. Performance Results

The performance of each algorithm was measured in milliseconds (ms):

Size	Dataset	Nodes	Edges	Tarjan (ms)	SCCs	TopoSort (ms)	DAGShortest (ms)
Small	small1-3	6-9	7-10	0.4-0.6	2-3	0.2	0.4
Medium	medium1-3	15-20	23-35	0.8-1.1	4-7	0.6-0.8	0.9
Large	large1-3	30-50	60-130	2.1-5.4	9-15	1.8-2.5	3.9

6. Screenshots





7. Conclusion

In this project, multiple advanced graph algorithms were successfully implemented and tested:

- Tarjan's algorithm efficiently found SCCs in linear time.
- Condensation Graph converted cyclic graphs into acyclic DAGs.
- Topological Sort provided a valid vertex order for DAG traversal.
- DAG Shortest Path efficiently calculated minimum distances.

The algorithms were validated through automated tests and performed well even on large graphs.

This project demonstrates how complex graph algorithms can be integrated for comprehensive graph analysis.

8. References

- R. Tarjan, "Depth-first search and linear graph algorithms", *SIAM Journal on Computing*, 1972.
- CLRS, *Introduction to Algorithms*, 3rd Edition.
- GeeksforGeeks: [Tarjan's Algorithm](#)